Laboratorio 01

DOCENTE	CARRERA	CURSO
D.Sc. Edgar Sarmiento Calisaya	Ciencia de la Computación	Ingeniería de Software II

LABORATORIO	TEMA	DURACIÓN
01	GIT	2 horas

1. Competencias del curso

- Aplica herramientas y entornos de ingeniería de software que contribuyan en la construcción y documentación del software, con especial énfasis en la comprensión de todo el proceso de desarrollo y los procesos de soporte. Esto debe incluir herramientas de construcción automática, automatización de pruebas e integración continua, incluyendo el control de cambios y versiones.
- Trabaja individualmente y en equipo, de manera ética y con compromiso, para el desarrollo de soluciones.

2. Competencias de la práctica

- Utiliza una herramienta de control de versiones.

3. Equipos y materiales

- Computadora
- Internet
- Material específico del curso descrito en el aula virtual: https://classroom.google.com/u/1/c/NzEyNjM2NTY1NzA2

4. Objetivo

- Controlar la evolución de un Proyecto de Software por medio de un Sistema de Control de Versiones (Git).

5. Prerrequisito

- Un proyecto de software (código fuente).
- Git: https://git-scm.com/download

6. Actividades

1. Instalar Git: https://git-scm.com/download

   sudo apt update #Actualizar paquetes:

   sudo apt-get install git #Instalar Git

   git –version #Verificar instalación

   #Configurar para persistir câmbios:

git config --global user.name "user"

git config --global user.email "user@mail.com"

2. Crear un repositorio local

   cd Project/

   git init #Inicializa el repositorio.

   git add . #Agrega todos los archivos al repositorio.

3. Realizar cambios

   git status #Consulta lo que ha cambiado

   git commit -a -m "commit message" #Guarda todos tus cambios

4. Crear ramas

   git branch feature

   git checkout feature

5. Hacer merges

   git checkout master

   git merge feature

6. Comparar dos ramas

   git diff master feature

   git diff master feature **--name-only** #solo nombres de archivos

   #otra forma

   git checkout feature

   git diff master

7. Revisar historial (commits y ramas)

   git status #Qué cambios están: staged, committed, or uncommitted

   git log

8. Regresar a la version anterior (HEAD es un puntero al último commit de la rama actual)

   git reset HEAD~1 # git reset --<soft/mixed/hard> HEAD~1

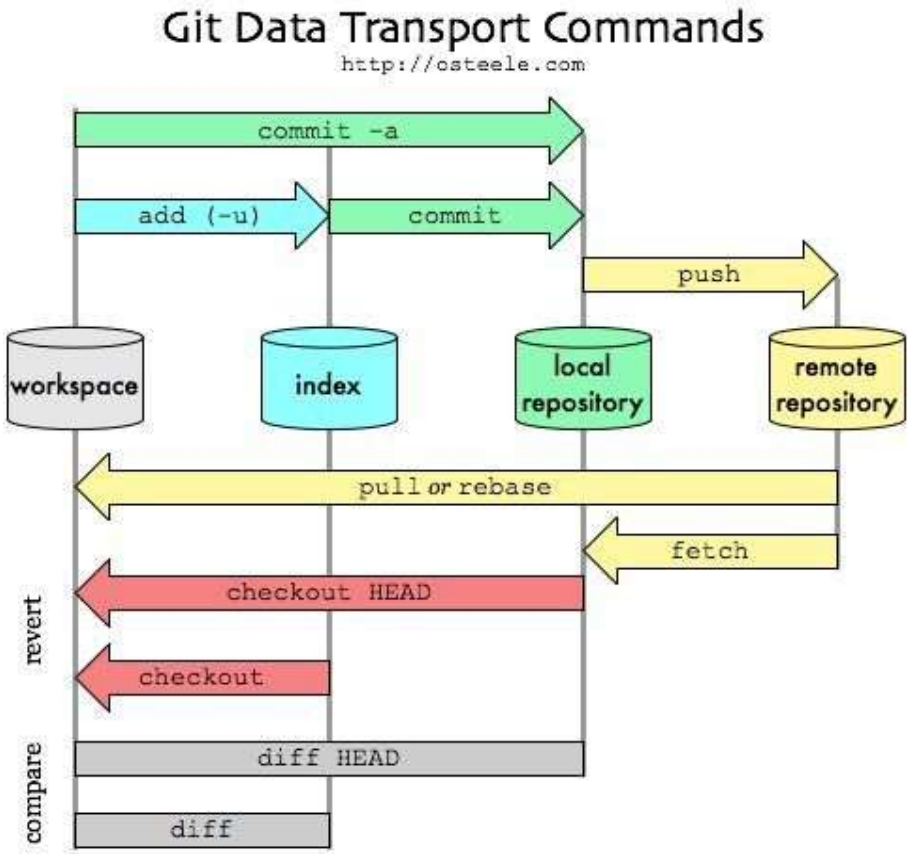
   git reset id_commit # regresar a un commit (por su ID) en específico

7. Entregables

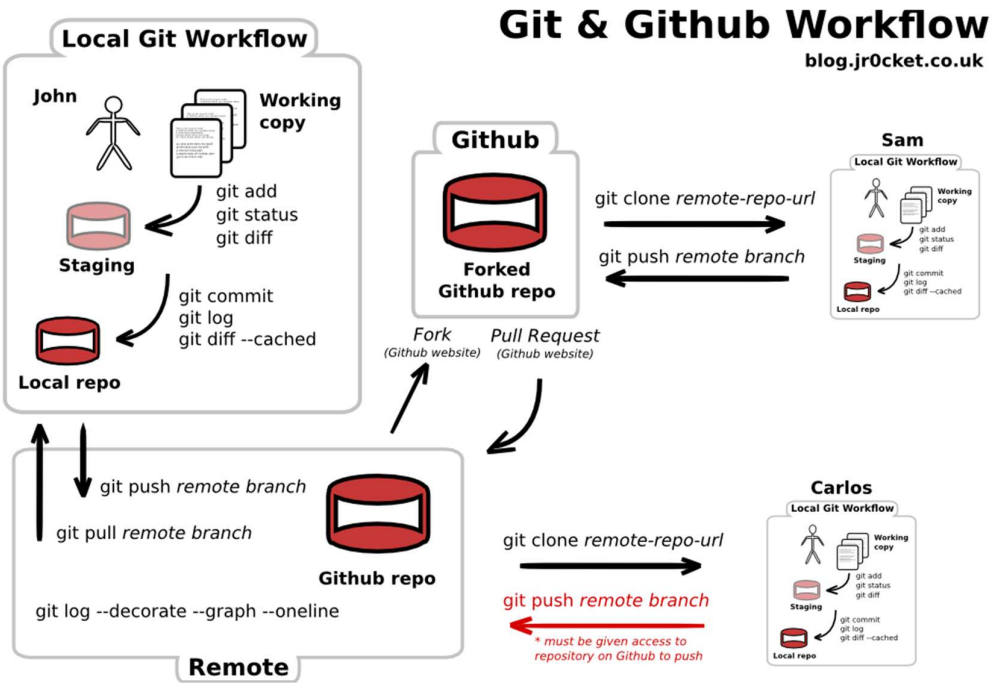
- Proyecto Git (commits, branchs, …): **git log**
- Documento con evidencias de ejecución de comandos GIT en su proyecto personal.

8. Conceptos y Documentación

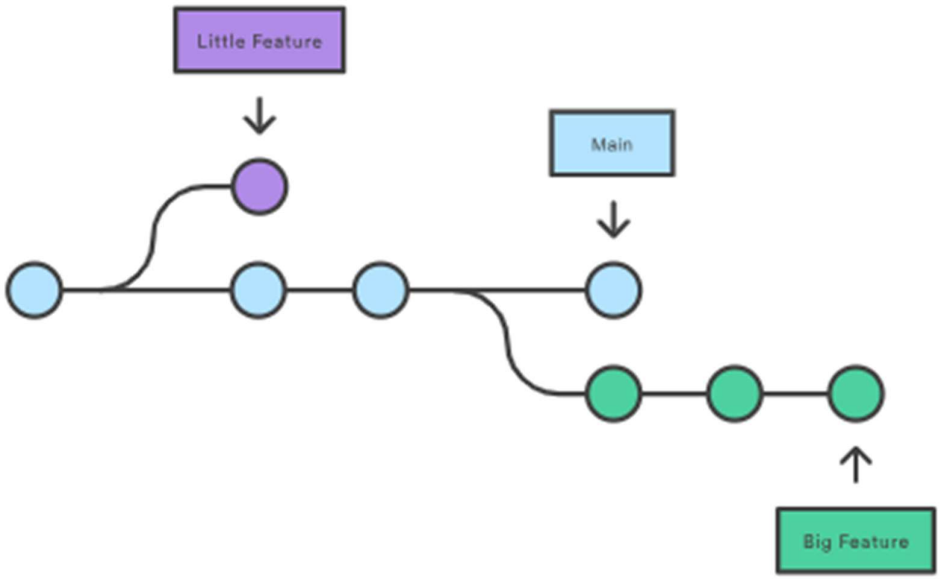
- Documentación Git: https://git-scm.com/doc
- **Git**: GIT es un sistema de control de versiones distribuido. Lo usaremos durante el curso para el trabajo semestral, así como para los ejemplos de los seminarios. GIT tiene varias docenas de comandos, pero solo necesitamos unos pocos. https://medium.com/@mehulgala77/git-fundamentals-history-architecture-popular-commands-1d3cd8bc1049



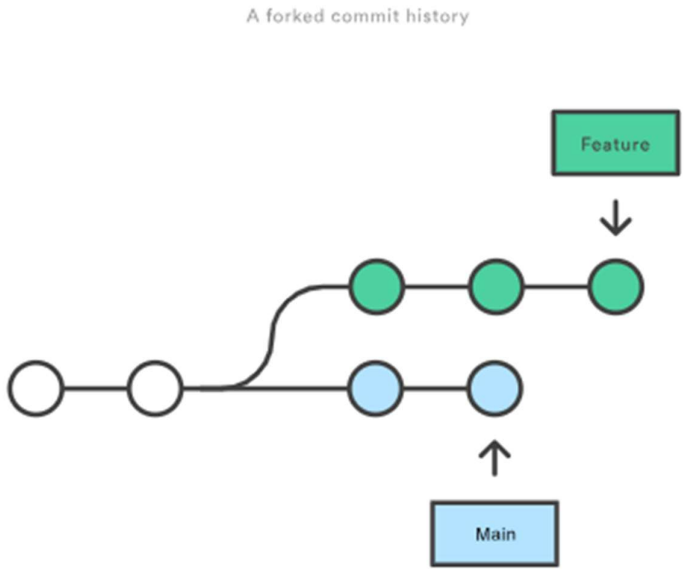
- Git and GitHub Integration Types: https://www.freecodecamp.org/news/the-beginners-guide-to-git-github/



- Git Branch: https://www.atlassian.com/br/git/tutorials/using-branches
    - https://gcapes.github.io/git-course/06-branching/index.html



- **Fusión de Dos Ramas (2 Escenarios): https://www.atlassian.com/git/tutorials/merging-vs-rebasing**



1. MERGE (Llevando los cambios de tu rama (**feature**) a la rama **main**.)

```
          A---B---C    feature
         /
D---E---F---G          master
```

```
git checkout master
git merge feature
```

```
          A---B---C    feature
         /           \
D---E---F---G--------H     master
```

2. REBASE (Trayendo cambios de la rama master a tu rama: Git quita los cambios que hiciste en tu rama (**feature**), luego trae los últimos cambios de **master**, y finalmente vuelve a poner los cambios de tu rama (**feature**) en la parte superior.)

```
          A---B---C    feature
         /
D---E---F---G          master
```

git checkout feature

git rebase master

```
                A'--B'--C' feature
               /
D---E---F---G master
```

- **Evitando Conflictos Innecesarios Durante la Fusión de Dos Ramas:** REGLAS PARA DISMINUIR CONFLICTOS:

1. Traer (a tu repositorio local) los últimos cambios de la rama master (remota).

   git checkout master

   git pull

2. Aplicar REBASE a tu rama (probablemente tendrás conflictos, pero serán mínimos y fáciles de resolver)

   git checkout feature

   git rebase master

Realice estos dos primeros pasos con frecuencia. Al menos una vez al día

3. Realice sus cambios en su rama (commits ...)

   git add .

   git commit -m "<<mensaje>>"

   ...

4. Cada vez que desee fusionar su rama con la rama master (u otra), asegúrate de hacer los dos primeros pasos justo antes y, finalmente, desde la master puede ejecutar <<git merge feature>>. Dado que su rama está totalmente actualizada con el master, Git simplemente pondrá su código en la parte superior de la pila master.

   git checkout master

   git merge feature

OBSERVACIONES:

1. El REBASE unifica las ramas perdiendo el historial de los commit y el merge no. Esto puede resultar bien importante cuando se necesite llevar o saber el historial de commit y se esta trabajando con otros compañeros en esa rama. Puedes llegar ser odiado por el equipo al usar rebase.

2. El comando REBASE dejar perder el historial, monta los commit encima de la otra rama sin importar de la cronología. Eso lo puedes observar en las imágenes de arriba.

REFERENCIAS:

Merging vs. rebasing: https://www.atlassian.com/git/tutorials/merging-vs-rebasing

Git Commands: https://git-scm.com/docs

DIFERENCIA ENTRE GIT REBASE Y GIT MERGE. https://medium.com/@MiguelCasas/diferencia-entre-git-rebase-y-git-merge-workshop-de-git-8622dedde2d7

Avoid 80% of Git merge conflicts: https://medium.com/front-end-weekly/avoid-80-of-merge-conflicts-with-git-rebase-b5d755a082a6